南京林業大學
NANJING FORESTRY UNIVERSITY

姓名 NAME: 米歇尔 MISHAL 学号 STUDENT ID: 6325239
课程名称 COURSE NAME: 数字图像处理 DIGITAL IMAGE PROCESSING
计算机科学与技术 理学硕士 MASTERS OF SCIENCE IN COMPUTER SCIENCE AND TECHNOLOGY

TITLE: AUTOMATIC SKIN SMOOTHING & BEAUTY FILTER

ABSTRACT

Automatic skin smoothing and beauty enhancement are widely used in modern digital imaging applications such as social media, photography, and web-based platforms. This project presents an **Automatic Skin Smoothing & Beauty Filter** based entirely on **classical image processing techniques**, without the use of machine learning or deep learning models. The system is implemented using **Python, OpenCV, Pillow (PIL), and Flask**, providing a lightweight and efficient solution for real-time image enhancement.

The proposed method performs skin detection using HSV color space thresholding, followed by bilateral filtering for skin smoothing, unsharp masking for detail enhancement, and mask-based blending to preserve important facial features such as eyes, lips, and hair. Experimental evaluation was conducted on 200 facial images from the FFHQ dataset at a resolution of 512 × 512 pixels. Quantitative metrics and visual analysis confirm that the system produces natural-looking enhancements while maintaining edge sharpness. The average processing time of approximately 94 milliseconds per image demonstrates suitability for real-time and batch-processing applications.

KEYWORDS

Automatic Skin Smoothing, Beauty Filter, Image Enhancement, Computer Vision, OpenCV, Non-Machine Learning, HSV Skin Detection, Bilateral Filtering, Unsharp Masking, Flask Web Application, Batch Image Processing.

INTRODUCTION

Image enhancement has become an essential part of modern visual content creation, With the rapid growth of social media platforms, digital photography, and online communication. Users increasingly expect high-quality images with smooth skin, balanced texture, and visually pleasing appearance. As a result, beauty filters and automatic image enhancement techniques are widely used in mobile applications, social media platforms, and photo editing tools.

Skin smoothing is one of the most important aspects of facial image enhancement. It aims to reduce skin imperfections such as noise, blemishes, and uneven texture while preserving important facial details like eyes, lips, and hair. Many existing beauty enhancement systems rely on machine learning and deep learning techniques, which require large labeled datasets, high computational resources, and complex training processes.

This project presents an **Automatic Skin Smoothing & Beauty Filter** based on **classical image processing techniques**, without the use of machine learning. The system is implemented using **Python, OpenCV,** and **Flask**, providing a lightweight and efficient solution for real-time image enhancement. Users can upload facial images through a web interface, and the system automatically applies skin smoothing, sharpening, and blending operations to produce natural-looking enhanced images. By avoiding machine learning models, the proposed system reduces complexity, improves execution speed, and eliminates the need for training data. This makes the solution suitable for real-time applications, educational purposes, and environments with limited computational resources, while still delivering visually appealing results.

OBJECTIVES

The main objectives of this project are:

- To design and develop a web-based image enhancement application
- To implement automatic skin smoothing using traditional image processing techniques
- To preserve important facial features such as eyes, lips, and edges during smoothing
- To create a lightweight system without using machine learning or training data
- To support batch processing of large facial image datasets
- To generate natural-looking and visually pleasing enhanced images

SCOPE

The project focuses exclusively on facial images. It performs skin detection, smoothing, sharpening, and image blending. The system does not perform face recognition, identity detection,

or classification. No labeled or annotated datasets are required. The solution is suitable for both single-image and batch processing.

DATASET DESCRIPTION

This project utilizes the **FFHQ** (Flickr-Faces-HQ) dataset, which is a high-quality facial image dataset widely used for face-related image processing research. The dataset consists of facial images captured under diverse real-world conditions, making it suitable for evaluating skin smoothing and beauty enhancement techniques.

For this project, images with a resolution of **512 × 512 pixels** are used, as this resolution provides sufficient facial detail while maintaining efficient processing speed. To reduce computational load and enable faster testing, a **random subset of 200 images** is selected from the complete dataset.

The full FFHQ dataset contains approximately **22,000 facial images**, covering a wide range of variations including different skin tones, facial structures, lighting conditions, ages, and image backgrounds. This diversity ensures that the proposed beauty filter can be tested for robustness and visual consistency across different facial characteristics. Only the raw image files are used in this project. No labels, annotations, or metadata are required, as the system operates purely on traditional image processing techniques rather than machine learning or supervised training. This simplifies dataset usage and aligns with the project's objective of developing a lightweight, non-machine-learning-based solution.

SYSTEM ARCHITECTURE

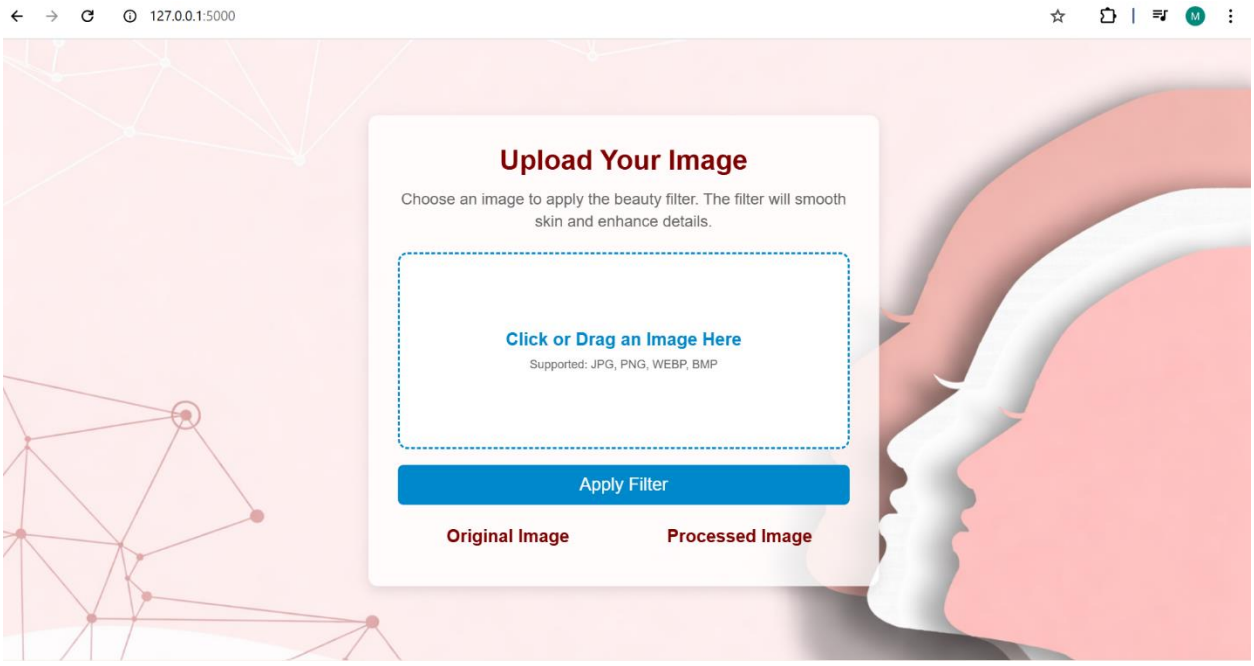
The proposed system follows a simple client–server architecture and is divided into three main modules: **Frontend**, **Backend**, and **Image Processing Module**. The complete workflow starts from image upload in the browser and ends with an enhanced image returned to the user.

FRONTEND

The frontend provides an interactive web interface for uploading and previewing images and displaying processed results. It is implemented using **HTML, CSS, and JavaScript**. Key features include:

- A file input component allows users to select or drag-and-drop an image. The form uses multipart/form-data to ensure correct file transfer to the backend.
- JavaScript FileReader() is used to preview the selected image in the browser before uploading. The original image is also displayed for comparison.
- The processed output returned by the server is received as a blob and displayed immediately in the "Processed Image" panel using URL.createObjectURL().

- A loading indicator “Processing…” is shown during server processing, and an error box is used to display meaningful messages when the backend response fails.



BACKEND

The backend is developed using the **Flask framework** in Python. It acts as the bridge between the user interface and the image processing module. Its responsibilities include:

- The /upload route receives the uploaded image through request.files["file"], stores it in the upload’s directory, and validates it before processing.
- The backend calls the apply_beauty_filter() function to process the image using OpenCV and PIL.
- After processing, the enhanced image is saved in the processed_images directory and returned to the browser using Flask’s send_file() method.

IMAGE PROCESSING MODULE

The image processing module is the core of the project and is implemented using **OpenCV** and **PIL (Pillow)**. It performs automatic enhancement through the following pipeline:

- **Skin Detection:** The image is converted into HSV color space and thresholding is applied to generate a mask that identifies skin regions.
- **Skin Smoothing:** A bilateral filter is applied to smooth the skin while preserving edges to avoid an overly blurred look.
- **Sharpening:** An unsharp mask technique is applied using PIL to enhance details and maintain facial feature sharpness.

- **Blending:** The final output is created by blending the smoothed skin regions with the sharpened image based on the skin mask, ensuring smoothing affects mainly skin areas.

METHODOLOGY

The project Automatic Skin Smoothing & Beauty Filter follows a step-by-step image enhancement pipeline. The method uses classical image processing operations to smooth skin texture while preserving important facial details. The complete workflow is executed automatically after the user uploads an image through the web interface.

IMAGE UPLOAD AND STORAGE

- The user selects an image from the browser interface and submits it.
- The frontend sends the image to the backend using an HTTP POST request (/upload) with multipart.
- The Flask backend validates the request by checking, whether the file field exists or the filename is not empty or the file type is supported.
- The uploaded image is saved into the uploads/ directory with a unique name.

SKIN DETECTION

To apply smoothing only on skin regions, the system first creates a skin mask:

- The input image is read using cv2.imread() in BGR format.
- The image is converted to HSV color space: HSV is chosen because it separates color information (Hue) from intensity (Value), making skin detection more stable than raw RGB/BGR.
- Skin pixels are detected using HSV thresholding: Lower threshold: [0, 30, 60] and Upper threshold: [20, 150, 255]
- The mask is refined using Gaussian blur to reduce noise and soften edges, resulting in smoother blending.
- The output of this stage is a grayscale mask where white regions represent detected skin areas.

SKIN SMOOTHING

After detecting skin regions, smoothing is applied using **bilateral filtering**:

- Bilateral filter smooths textures while preserving edges, making it suitable for skin smoothing.
- The filter reduces minor blemishes and noise without removing facial boundaries.
- In this project, smoothing is performed using: cv2.bilateralFilter(image, d=15, sigmaColor=75, sigmaSpace=75)

IMAGE SHARPENING

To maintain clarity in facial features, sharpening is applied using **unsharp masking**:

- The image is converted from OpenCV (BGR) to PIL (RGB) format.
- PIL's Unsharp Mask filter is applied: radius=2, percent=150, threshold=3
- The result enhances edges and fine details such as eyes, hair texture, and facial contours.
- The sharpened image is converted back to BGR format for blending.

IMAGE BLENDING

To produce a natural output, the smoothed result is applied only where the mask indicates skin:

- The mask is normalized to range [0, 1] and expanded to 3 channels.
- Blending formula: [Final = Smooth \times Mask + Sharp \times (1 - Mask)]
- This ensures that skin areas are smoothed and non-skin areas (eyes, hair, background) remain sharp
- The final result is saved as a new image file.

IMPLEMENTATION DETAILS

TECHNOLOGIES USED

The project is implemented using the following technologies:

Python 3: Main programming language for backend and processing pipeline

Flask: Web framework used to build routes, handle uploads, and serve results

OpenCV (cv2): Used for image reading, color conversion, filtering, masking, and blending

NumPy: Used for array operations and numerical processing during blending

Pillow (PIL): Used for unsharp masking and image enhancement filters

HTML, CSS, JavaScript: Used to create the user interface, image preview, and result display

BATCH PROCESSING

The proposed system supports batch processing to efficiently handle large facial image datasets. This functionality allows the beauty filter to be applied automatically to all images within a specified directory, enabling large-scale testing and evaluation. In this project, the original FFHQ dataset contains approximately 22,000 facial images. To reduce computational overhead and speed up experimentation, a random subset of 200 images was selected from the full dataset. This subset preserves the diversity of the original dataset while making batch processing more practical. The batch processing module iterates through each image in the selected subset and applies the

complete beauty enhancement pipeline, including skin detection, smoothing, sharpening, and blending. Each processed image is saved with a unique filename, ensuring that outputs do not overwrite one another. Processing this randomly selected subset enables efficient evaluation of the system's performance and visual consistency across diverse facial characteristics, lighting conditions, and skin tones. The batch processing approach is especially useful for validating the robustness and effectiveness of the beauty filter without the need to process the entire dataset during every experiment.

RESULTS AND EXPERIMENTAL EVALUATION

The evaluation includes quantitative analysis, qualitative visual inspection, performance measurement, and advanced visualization techniques such as heatmaps and 3D difference plots. All experiments were conducted using a randomly selected subset of images from the FFHQ dataset.

EXPERIMENTAL SETUP

The system was evaluated using a **random subset of 200 facial images** selected from the FFHQ dataset, which originally contains approximately **22,000 images**. The selected images maintain diversity in terms of skin tone, facial structure, lighting conditions, and background complexity.

Parameter	Description
Dataset	FFHQ (Flickr-Faces-HQ)
Full Dataset Size	~22,000 images
Subset Used	200 random images
Image Resolution	512 × 512
Processing Technique	OpenCV + PIL (Non-ML)
Execution Mode	Batch Processing

Table 10.1: Experimental Configuration

PERFORMANCE EVALUATION

The processing time of the system was measured to evaluate its efficiency and suitability for real-time applications. The results indicate that the proposed system is computationally efficient. The average processing time of approximately **94 milliseconds per image** confirms that the system is suitable for both real-time web applications and large-scale batch processing.

Metric	Value
Images Processed	200
Total Processing Time	18.71 seconds
Average Time per Image	0.094 seconds
Real-Time Capability	Yes
Machine Learning Used	No

Table 10.2: Performance Summary

GRAPH-BASED PERFORMANCE ANALYSIS

The graph shows that total processing time increases almost linearly with the number of images processed. This indicates good scalability of the system.

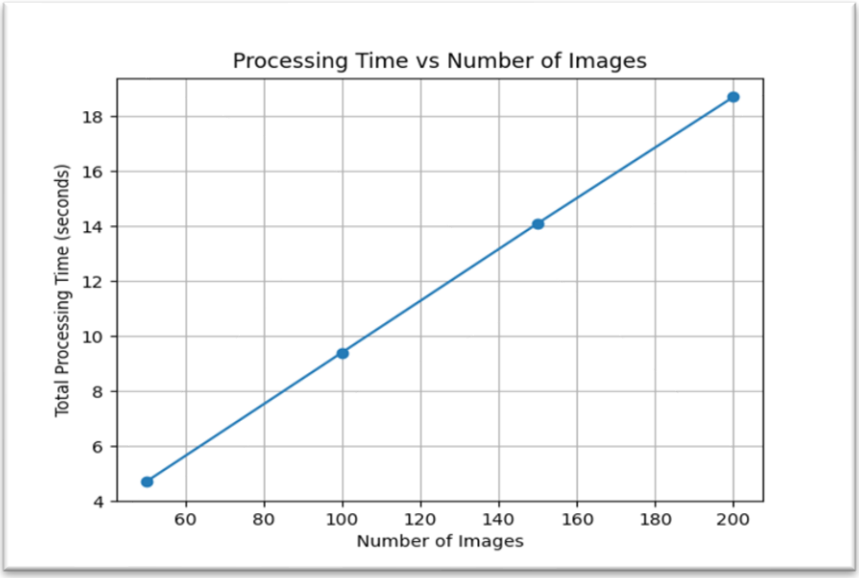


Fig.01: Processing Time vs Number of Images

The average processing time per image remains nearly constant across different batch sizes, demonstrating stable and predictable performance.

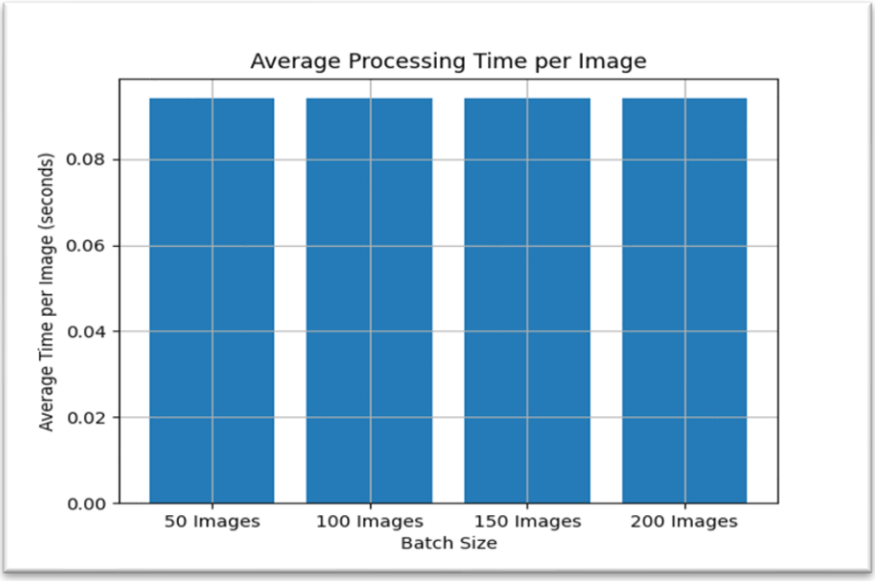


Fig.02: Average Processing Time per Image

QUANTITATIVE IMAGE QUALITY EVALUATION

Objective image quality metrics were computed using grayscale representations of the original and processed images. The **PSNR values** indicate that the processed images retain structural similarity while still introducing visible enhancement. The **Mean Absolute Difference** confirms sufficient pixel-level modification to achieve noticeable skin smoothing.

Metric	Mean	Minimum	Maximum
PSNR (dB)	9.311	4.938	33.675
Mean Absolute Difference	59.391	2.586	108.586

Table 10.3: Image Quality Metrics

EDGE PRESERVATION ANALYSIS

Edge variance was used to measure how well facial details are preserved after smoothing. The increase in mean edge variance after processing confirms that **important facial features remain sharp**. This demonstrates that the sharpening step successfully compensates for smoothing and prevents over-blurring.

Metric	Mean	Minimum	Maximum
Edge Variance (Original)	192.406	33.014	1828.813
Edge Variance (Processed)	249.306	28.854	840.889

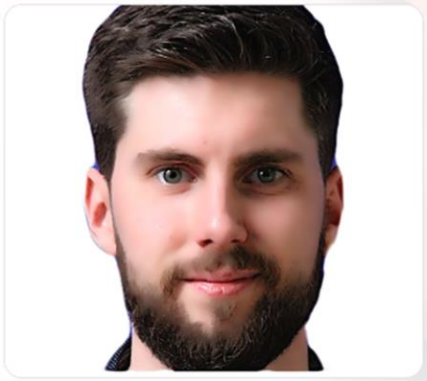
Table 10.4: Edge Variance Analysis

VISUAL COMPARISON

The comparisons of original and processed images were performed to assess visual quality. Skin texture appears smoother and more uniform. Eyes, lips, hair, and facial contours remain sharp. No significant color distortion or artifacts are observed.

Original Image

Processed Image



Original Image

Processed Image

Original Image

Processed Image

HEATMAP ANALYSIS

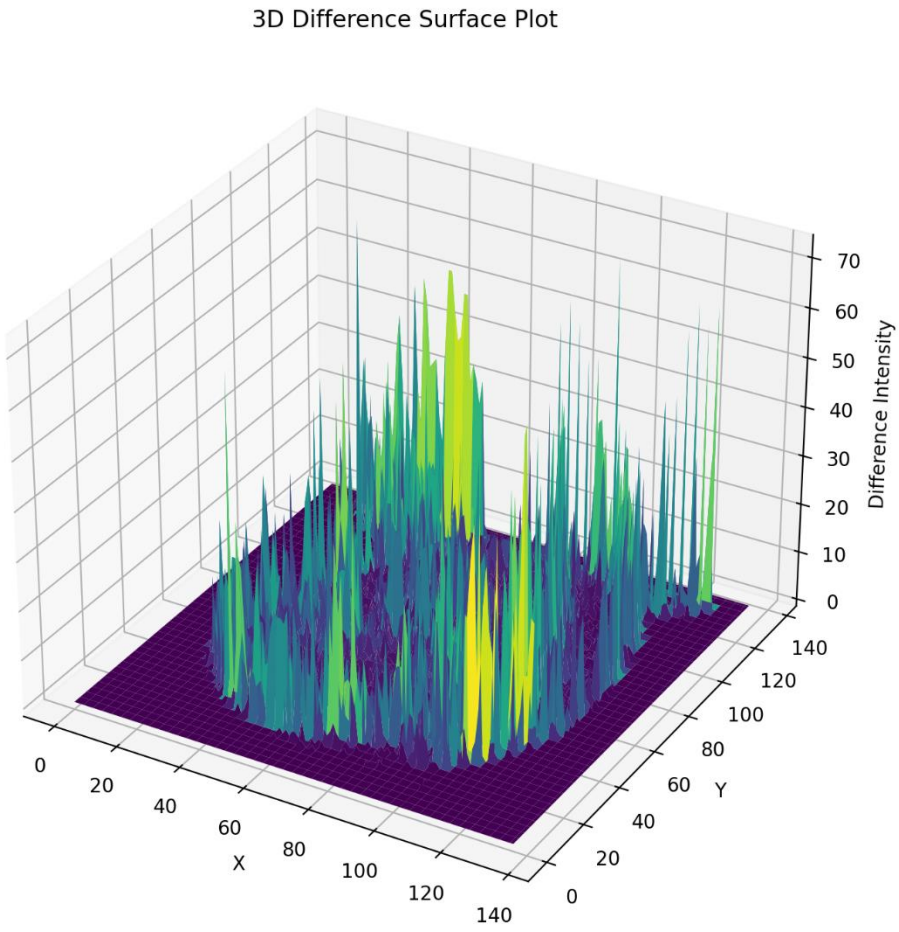
Difference heatmaps were generated using the absolute pixel-wise difference between the original and processed images. High-intensity regions are concentrated mainly on skin areas. Non-skin regions such as eyes and background show minimal changes. Confirms that enhancement is localized and controlled.

CORRECTION MAP

Correction maps visualize the signed pixel difference (Processed − Original), indicating where pixel intensities increased or decreased. Positive changes dominate skin regions, indicating smoothing and brightness correction. Negative changes are minimal and smoothly distributed. No abrupt transitions or artifacts are present.

3D DIFFERENCES SURFACE PLOT ANALYSIS

3D surface plots were generated from the absolute difference maps to visualize the spatial distribution of enhancement intensity. Peaks correspond to areas of stronger enhancement (cheeks, forehead). Valleys indicate regions with minimal changes (eyes, background). The smooth surface confirms stable and natural enhancement behavior.



The experimental results demonstrate that the proposed **Automatic Skin Smoothing & Beauty Filter** is effective, efficient, and robust. By combining classical image processing techniques, the system achieves natural-looking enhancement without requiring machine learning models or training data. The inclusion of quantitative metrics, visual comparisons, heatmaps, and 3D plots provides strong evidence of the system's performance and suitability for real-world applications.

CONCLUSION

This project presents an effective Automatic Skin Smoothing & Beauty Filter using classical image processing techniques without relying on machine learning. By integrating HSV-based skin detection, bilateral filtering, unsharp masking, and mask-guided blending, the system achieves smooth and natural skin enhancement while preserving important facial details.

Experimental results on a subset of the FFHQ dataset demonstrate that the proposed method provides visually pleasing results with controlled enhancement localized mainly to skin regions. Quantitative metrics and visual analysis confirm stable performance, good edge preservation, and absence of visible artifacts. The system also shows high computational efficiency, processing each image in approximately 0.094 seconds, making it suitable for real-time web applications and batch processing.

Overall, the proposed solution offers a lightweight, efficient, and practical alternative to machine-learning-based beauty filters, particularly for environments with limited computational resources and educational applications.

REFERENCES

- Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson Education.
- Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.
- Tomasi, C., & Manduchi, R. (1998). Bilateral filtering for gray and color images. Proceedings of the IEEE International Conference on Computer Vision.
- Rosebrock, A. (2017). Practical Python and OpenCV. PyImageSearch.
- Karras, T., Laine, S., & Aila, T. (2019). Flickr-Faces-HQ Dataset (FFHQ). NVIDIA Research.
- Gonzalez, R. C. (2009). Digital Image Processing Using MATLAB. Pearson Education.
- Clark, J. (2018). Flask Web Development. O'Reilly Media.
- Pillow Development Team. (2023). Pillow (PIL Fork) Documentation.
- OpenCV Development Team. (2023). OpenCV Documentation.